



Universidade Federal de Santa Catarina
Centro Tecnológico - CTC - Sistemas de Informação
INE5645-05238A/B - Programação Paralela e Distribuída
Professor Odorico Machado Mendizabal
Camila Prim - 23203397

INTRODUÇÃO

Este projeto apresenta um sistema de vendas paralelo desenvolvido em **Python** para demonstrar conceitos de programação concorrente e paralela. A aplicação simula o processamento de pedidos de uma loja, passando pelas etapas de criação, validação, análise financeira e logística.

A arquitetura segue o padrão **pipeline**, em que cada pedido percorre uma sequência de etapas. Essas etapas são executadas por processos independentes, chamados **workers**, permitindo que vários pedidos sejam processados simultaneamente. A comunicação entre os processos ocorre por meio de filas do módulo **multiprocessing**, que funcionam como buffers entre produtores e consumidores.

Além do fluxo principal, o sistema possui um monitor executado em processo separado, responsável por registrar os eventos da execução, gerar logs e produzir um relatório final em JSON. O código foi dividido em módulos específicos para configuração, modelo de pedido, **workers**, monitoramento, orquestração e inicialização da aplicação.

Assim, o projeto demonstra de forma prática o uso de processos, filas, **worker pools**, **pipeline** e monitoramento em um cenário simulado de vendas.

OBJETIVO

O objetivo deste projeto é implementar um sistema que simula um ambiente em que vários clientes criam pedidos ao mesmo tempo. Esses pedidos são enviados para uma fila de validação, processados por validadores, encaminhados para análise financeira e, caso aprovados, enviados para a logística. Durante todo o processo, um monitor registra os eventos gerados pelos **workers** e produz um relatório final com os resultados da execução.

Entre os principais objetivos técnicos do projeto estão:

- Aplicar o padrão Produtor/Consumidor
- Utilizar Worker Pools para processar várias tarefas em paralelo
- Simular um pipeline de processamento de pedidos
- Registrar eventos de execução em arquivo de log
- Gerar um relatório final com estatísticas do sistema
- Controlar corretamente a inicialização e finalização dos processos

CONCEITOS DE PROGRAMAÇÃO CONCORRENTE UTILIZADOS

Processo

O sistema utiliza processos independentes por meio da classe **Process** do módulo **multiprocessing**. Cada worker é executado em um processo separado, permitindo que diferentes etapas trabalhem simultaneamente.

No orquestrador, cada cliente, validador, financeiro e logístico é criado como um processo separado. Esses processos são armazenados em uma lista para que o sistema consiga aguardar sua finalização posteriormente.

Fila

As filas são usadas para comunicação entre os processos. Como processos não compartilham memória da mesma forma que threads, o sistema utiliza Queue para transportar pedidos de uma etapa para outra.

O projeto possui três filas principais:

- **fila_pedidos** → recebe pedidos criados pelos clientes
- **fila_validados** → recebe pedidos aprovados na validação
- **fila_aprovados** → recebe pedidos aprovados financeiramente

Produtor/Consumidor

O padrão Produtor/Consumidor aparece quando um processo produz dados e outro processo consome esses dados.

No projeto:

- Clientes produzem pedidos.
- Validadores consomem pedidos e produzem pedidos validados.
- Financeiros consomem pedidos validados e produzem pedidos aprovados.
- Logísticos consomem pedidos aprovados e finalizam o fluxo.

Worker Pool

O sistema usa vários workers para a mesma etapa. Por exemplo, pode haver dois validadores, dois financeiros e dois logísticos. Isso caracteriza um Worker Pool, pois vários processos executam a mesma função e consomem tarefas de uma mesma fila.

A quantidade de workers é configurável no arquivo **config.py**, por meio dos atributos **num_validadores**, **num_financeiros** e **num_logisticos**.

Pipeline

O pipeline é o fluxo em etapas. Um pedido não vai diretamente do cliente para a entrega. Ele precisa passar por uma sequência:

Criação → Validação → Financeiro → Logística → Finalização

Cada etapa recebe o resultado da etapa anterior.

Active Object

O monitor funciona como um objeto ativo, porque ele possui sua própria fila de mensagens e executa em um processo separado. Os workers não escrevem diretamente o relatório final, eles enviam eventos para o monitor, e o monitor centraliza o registro desses eventos .

FLUXO DE EXECUÇÃO DE UM PEDIDO

O ciclo de vida de um pedido começa no **cliente_worker**. Cada cliente gera uma quantidade definida de pedidos, escolhendo aleatoriamente um item, um nome de cliente e um valor. Após ser criado, o pedido é colocado na fila inicial do sistema, chamada **fila_pedidos**.

Depois disso, um dos validadores disponíveis retira o pedido da fila. O validador simula um tempo de processamento e decide, com base em uma taxa de falha configurada, se o pedido será rejeitado ou aprovado. Se o pedido for rejeitado, ele recebe o status **REJEITADO_VALIDACAO** e é registrado como finalizado. Se for aprovado, recebe o status **VALIDADO** e é enviado para a próxima fila.

Na etapa financeira, um processo financeiro consome pedidos da fila de validados. Assim como na validação, existe uma simulação de tempo e uma possibilidade de falha. Se o pedido for recusado, ele recebe o status **REJEITADO_FINANCEIRO** e é finalizado. Se for aprovado, recebe o status **APROVADO_FINANCEIRO** e segue para a fila de pedidos aprovados.

Por fim, a logística consome os pedidos aprovados financeiramente. Nessa etapa, o sistema simula a tentativa de entrega. O pedido pode receber o status **ENTREGUE** ou **FALHA_ENTREGA**. Em ambos os casos, o pedido é considerado finalizado e enviado ao monitor para registro final.

Os status possíveis de um pedido estão definidos no enum `StatusPedido`, dentro do arquivo `pedido.py`. Esse arquivo também define quais status representam o fim do processamento, como rejeição na validação, rejeição financeira, entrega concluída ou falha na entrega.

EXPLICAÇÃO DOS MÓDULOS

`config.py`

O arquivo `config.py` define a classe `ConfiguracaoSistema` e é responsável por concentrar parâmetros de execução.

Os principais parâmetros definidos são:

- **num_validadores**
- **num_financeiros**
- **num_logisticos**
- **taxa_falha_validacao**
- **taxa_falha_financeira**
- **taxa_falha_logistica**
- **num_clientes**
- **pedidos_por_cliente**
- **tamanho_fila**
- **tempo_processamento_min**
- **tempo_processamento_max**
- **arquivo_log**

Esses parâmetros permitem alterar o comportamento do sistema sem modificar diretamente a lógica dos workers. Por exemplo, para aumentar a carga do sistema, basta aumentar o número de clientes ou a quantidade de pedidos por cliente. Para testar maior paralelismo, apenas precisa aumentar a quantidade de validadores, financeiros ou logísticos. E além de ter uma lista configurável de itens e nomes dos clientes para enfeitar o trabalho.

```
@dataclass
class ConfiguracaoSistema:
    # quantos workers por etapa
    num_validadores: int = 2
    num_financeiros: int = 2
    num_logisticos: int = 2

    # taxas de falha
    taxa_falha_validacao: float = 0.25
    taxa_falha_financeira: float = 0.25
    taxa_falha_logistica: float = 0.25

    # define quantos clientes e pedidos
    num_clientes: int = 4
    pedidos_por_cliente: int = 6

    # fila limitada
    tamanho_fila: int = 10

    # tempos de processamento em segundos
    tempo_processamento_min: float = 0.5
    tempo_processamento_max: float = 2.0

    arquivo_log: str = "sistema_vendas.log"
```

```
itens_disponiveis: List[str] = field(default_factory=lambda: [
    "Notebook",
    "Mouse",
    "Teclado",
    "Monitor",
    "Webcam",
    "Headphone",
    "SSD",
    "Memória RAM"
])

nomes_clientes: List[str] = field(default_factory=lambda: [
    "João Silva",
    "Maria Santos",
    "Pedro Oliveira",
    "Ana Costa"
])
```

O arquivo também possui validações para garantir que os valores configurados sejam válidos. por exemplo, o número de clientes, workers, tamanho da fila precisam ser maiores que zero. As taxas de falha precisam estar entre 0.0 e 1.0. Isso evita que o sistema seja iniciado com configurações inválidas.

Também existem configurações prontas, como **CONFIG_PADRAO**, **CONFIG_TESTE**, **CONFIG_PESADO**, que podem ser usadas para diferentes cenários de execução

```
def validar(self):
    """Valida configurações para falhar cedo, antes de iniciar processos."""
    if self.num_clientes <= 0:
        raise ValueError("num_clientes deve ser maior que zero")
    if self.pedidos_por_cliente <= 0:
        raise ValueError("pedidos_por_cliente deve ser maior que zero")
    if self.num_validadores <= 0:
        raise ValueError("num_validadores deve ser maior que zero")
    if self.num_financeiros <= 0:
        raise ValueError("num_financeiros deve ser maior que zero")
    if self.num_logisticos <= 0:
        raise ValueError("num_logisticos deve ser maior que zero")
    if self.tamanho_fila <= 0:
        raise ValueError("tamanho_fila deve ser maior que zero")
    if self.tempo_processamento_min < 0:
        raise ValueError("tempo_processamento_min não pode ser negativo")
    if self.tempo_processamento_max < self.tempo_processamento_min:
        raise ValueError("tempo_processamento_max deve ser maior ou igual ao minimo")

    taxas = {
        "taxa_falha_validacao": self.taxa_falha_validacao,
        "taxa_falha_financeira": self.taxa_falha_financeira,
        "taxa_falha_logistica": self.taxa_falha_logistica,
    }
    for nome, valor in taxas.items():
        if not 0.0 <= valor <= 1.0:
            raise ValueError(f"{nome} deve estar entre 0.0 e 1.0")

def total_pedidos_esperados(self) -> int:
    """Calcula total de pedidos que serão gerados."""
    return self.num_clientes * self.pedidos_por_cliente
```

```
#configuração padrão
CONFIG_PADRAO = ConfiguracaoSistema()

#configuração para testes rápidos
CONFIG_TESTE = ConfiguracaoSistema(
    num_validadores=2,
    num_financeiros=1,
    num_logisticos=1,
    pedidos_por_cliente=10,
    tamanho_fila=10,
    tempo_processamento_min=0.1,
    tempo_processamento_max=0.5,
    arquivo_log="sistema_vendas.log"
)

# configuração para teste pesado
CONFIG_PESADO = ConfiguracaoSistema(
    num_validadores=5,
    num_financeiros=4,
    num_logisticos=3,
    pedidos_por_cliente=200,
    tamanho_fila=50,
    tempo_processamento_min=0.5,
    tempo_processamento_max=2.0,
    arquivo_log="sistema_vendas.log"
)
```

pedido.py

O módulo define a estrutura dos pedidos processados pelo sistemas. O pedido é representado pela classe Pedido, que armazena informações como identificador, cliente, item, valor, status e timestamps.

Cada pedido possui um status, definido pelo enum **StatusPedido**. Esse enum representa os estados possíveis durante o fluxo:

- **NOVO**
- **VALIDADO**
- **REJEITADO_VALIDACAO**
- **APROVADO_FINANCEIRO**
- **REJEITADO_FINANCEIRO**
- **ENTREGUE**
- **FALHA_ENTREGA**

```
class StatusPedido(Enum):
    NOVO = "Novo"
    VALIDADO = "Validado"
    REJEITADO_VALIDACAO = "Rejeitado na Validação"
    APROVADO_FINANCEIRO = "Aprovado Financeiramente"
    REJEITADO_FINANCEIRO = "Rejeitado Financeiramente"
    ENTREGUE = "Entregue"
    FALHA_ENTREGA = "Falha na Entrega"

    STATUS_FINAIS = {
        StatusPedido.REJEITADO_VALIDACAO,
        StatusPedido.REJEITADO_FINANCEIRO,
        StatusPedido.ENTREGUE,
        StatusPedido.FALHA_ENTREGA,
    }
```

A classe também possui métodos importantes, como

atualizar_status(), que altera o status do pedido e registra automaticamente um timestamp da etapa correspondente. Isso permite acompanhar em quais momentos o pedido passou por cada fase do pipeline.

Outro método importante é **esta_finalizado()**, que verifica se o pedido chegou a um status final. Os status finais são aqueles em que o pedido não precisa seguir para mais nenhuma etapa, como rejeição, entrega ou falha na entrega.

A função **criar_pedido()** é responsável por gerar um novo pedido com um identificador único baseado em uuid.

```

@dataclass
class Pedido:
    id: str
    cliente_id: str
    nome_cliente: str
    item: str
    valor: float
    status: StatusPedido = StatusPedido.NOVO
    timestamp_criacao: datetime = field(default_factory=datetime.now)
    timestamps: Dict[str, datetime] = field(default_factory=dict)

    def registrar_timestamp(self, etapa: str):
        self.timestamps[etapa] = datetime.now()

    def atualizar_status(self, novo_status: StatusPedido):
        self.status = novo_status
        etapa = novo_status.value.lower().replace(" ", "_")
        self.registrar_timestamp(etapa)

    def esta_finalizado(self) -> bool:
        return self.status in STATUS_FINAIS

    def para_dict(self) -> dict:
        return {
            "id": self.id,
            "cliente_id": self.cliente_id,
            "nome_cliente": self.nome_cliente,
            "item": self.item,
            "valor": self.valor,
            "status": self.status.value,
            "timestamp_criacao": self.timestamp_criacao.isoformat(),
            "timestamps": {k: v.isoformat() for k, v in self.timestamps.items()}
        }

    def __str__(self):
        return (
            f"Pedido({self.id}, Cliente={self.nome_cliente}, "
            f"Item={self.item}, Status={self.status.value})"
        )

def criar_pedido(cliente_id: str, nome_cliente: str, item: str, valor: float, config=None) -> Pedido:
    pedido_id = str(uuid.uuid4())[:8]

    return Pedido(id=pedido_id, cliente_id=cliente_id, nome_cliente=nome_cliente, item=item, valor=valor)

```

workers.py

Esse arquivo contém as funções executadas pelos processos do sistemas. Cada função representa um tipo de trabalhador dentro do pipeline.

Os workers são:

- cliente_worker
- validador_worker
- financeiro_worker
- logistica_worker

O **cliente_worker()** é responsável por gerar pedidos. Ele sorteia um item disponível, um nome de cliente e um valor. Depois cria um objeto Pedido e o envia para a fila de saída. Essa fila será consumida pelos validadores

```

def cliente_worker(cliente_id: int, config: ConfiguracaoSistema, fila_saida: Queue, fila_monitor: Optional[Queue] = None):
    try:
        for num_pedido in range(config.pedidos_por_cliente):
            item = random.choice(config.itens_disponiveis)
            nome_cliente = random.choice(config.nomes_clientes)
            valor = round(random.uniform(100, 5000), 2)

            pedido = criar_pedido(cliente_id=f"CLT-{cliente_id:03d}", nome_cliente=nome_cliente, item=item, valor=valor, config=config)

            fila_saida.put(pedido)

            detalhes = {
                "cliente": pedido.nome_cliente,
                "item": item,
                "valor": f"R$ {valor:.2f}",
                "pedido": f"{num_pedido + 1}/{config.pedidos_por_cliente}"
            }

            registrar_evento(fila_monitor, etapa="produtor", worker_id=cliente_id, pedido_id=pedido.id, evento="pedido_criado", detalhes=detalhes)
            log("PROD", cliente_id, f"CRIOU pedido {pedido.id} - {detalhes}")

            time.sleep(random.uniform(0.05, 0.15))

        log("PROD", cliente_id, "LIBERADO")

    except Exception as exc:
        registrar_erro(fila_monitor, f"cliente_worker {cliente_id}: {exc!r}")
        log("PROD", cliente_id, f"ERRO: {exc!r}")
        raise

```

O **validador_worker()** consome pedidos da fila inicial. Para cada pedido, ele registra o início da validação, simula um tempo de processamento e decide se o pedido será aprovado ou rejeitado. Caso seja rejeitado, o pedido é finalizado. Caso seja validado, é colocado na fila de pedidos validados.

```
def validador_worker(worker_id: int, config: ConfiguracaoSistema, fila_entrada: Queue, fila_saida: Queue, fila_monitor: Optional[Queue] = None):
    try:
        while True:
            pedido: Pedido = fila_entrada.get()

            if pedido is None:
                break

            log("VAL", worker_id, f"INICIANDO pedido {pedido.id} - {pedido.nome_cliente}")
            registrar_evento(fila_monitor, etapa="validacao", worker_id=worker_id, pedido_id=pedido.id, evento="inicio_validacao")

            tempo_processamento(config)

            if random.random() < config.taxa_falha_validacao:
                pedido.atualizar_status(StatusPedido.REJEITADO_VALIDACAO)
                registrar_evento(fila_monitor, etapa="validacao", worker_id=worker_id, pedido_id=pedido.id, evento="pedido_rejeitado_validacao")

                registrar_pedido_finalizado(fila_monitor, pedido)
                log("VAL", worker_id, f"FINALIZOU pedido {pedido.id} - REJEITADO")
            else:
                pedido.atualizar_status(StatusPedido.VALIDADO)
                registrar_evento(fila_monitor, etapa="validacao", worker_id=worker_id, pedido_id=pedido.id, evento="pedido_validado")
                fila_saida.put(pedido)
                log("VAL", worker_id, f"FINALIZOU pedido {pedido.id} - OK")

            log("VAL", worker_id, "LIBERADO")

    except Exception as exc:
        registrar_erro(fila_monitor, f"validador_worker {worker_id}: {exc!r}")
        log("VAL", worker_id, f"ERRO: {exc!r}")
        raise
```

O **financeiro_worker()** consome pedidos já validados. Ele simula uma análise financeira e pode aprovar ou rejeitar o pedido. Pedidos rejeitados são finalizados. Pedidos aprovados são encaminhados para a fila de pedidos aprovados.

```
def financeiro_worker(worker_id: int, config: ConfiguracaoSistema, fila_entrada: Queue, fila_saida: Queue, fila_monitor: Optional[Queue] = None):
    try:
        while True:
            pedido: Pedido = fila_entrada.get()

            if pedido is None:
                break

            log("FIN", worker_id, f"INICIANDO pedido {pedido.id}")
            registrar_evento(fila_monitor, etapa="financeiro", worker_id=worker_id, pedido_id=pedido.id, evento="inicio_financeiro")

            tempo_processamento(config)

            if random.random() < config.taxa_falha_financeira:
                pedido.atualizar_status(StatusPedido.REJEITADO_FINANCEIRO)
                registrar_evento(fila_monitor, etapa="financeiro", worker_id=worker_id, pedido_id=pedido.id, evento="pedido_rejeitado_financeiro")

                registrar_pedido_finalizado(fila_monitor, pedido)
                log("FIN", worker_id, f"FINALIZOU pedido {pedido.id} - RECUSADO")
            else:
                pedido.atualizar_status(StatusPedido.APROVADO_FINANCEIRO)
                registrar_evento(fila_monitor, etapa="financeiro", worker_id=worker_id, pedido_id=pedido.id, evento="pedido_aprovado_financeiro")

                fila_saida.put(pedido)
                log("FIN", worker_id, f"FINALIZOU pedido {pedido.id} - OK")

            log("FIN", worker_id, "LIBERADO")

    except Exception as exc:
        registrar_erro(fila_monitor, f"financeiro_worker {worker_id}: {exc!r}")
        log("FIN", worker_id, f"ERRO: {exc!r}")
```

O **logistica_worker()** consome pedidos aprovados financeiramente. Ele simula o processo de entrega e define se o pedido foi entregue com sucesso ou se houve falha. Em qualquer um dos casos, o pedido é finalizado e registrado no monitor.

```
def logistica_worker(worker_id: int, config: ConfiguracaoSistema, fila_entrada: Queue, fila_monitor: Optional[Queue] = None):
    try:
        while True:
            pedido = fila_entrada.get()

            if pedido is None:
                break

            log("LOG", worker_id, f"INICIANDO entrega do pedido {pedido.id}")
            registrar_evento(fila_monitor, etapa="logistica", worker_id=worker_id, pedido_id=pedido.id, evento="inicio_logistica")

            tempo_processamento(config)

            if random.random() < config.taxa_falha_logistica:
                pedido.atualizar_status(StatusPedido.FALHA_ENTREGA)
                registrar_evento(fila_monitor, etapa="logistica", worker_id=worker_id, pedido_id=pedido.id, evento="falha_entrega")

                log("LOG", worker_id, f"FINALIZOU pedido {pedido.id} - FALHA")
            else:
                pedido.atualizar_status(StatusPedido.ENTREGUE)
                registrar_evento(fila_monitor, etapa="logistica", worker_id=worker_id, pedido_id=pedido.id, evento="pedido_entregue")
                log("LOG", worker_id, f"FINALIZOU pedido {pedido.id} - ENTREGUE")

            registrar_pedido_finalizado(fila_monitor, pedido)

        log("LOG", worker_id, "LIBERADO")

    except Exception as exc:
        registrar_erro(fila_monitor, f"logistica_worker {worker_id}: {exc!r}")
        log("LOG", worker_id, f"ERRO: {exc!r}")
        raise
```

Todas as funções possuem tratamento de erro. Caso ocorra uma exceção, o erro é registrado no monitor e exibido no terminal. Depois disso, a exceção é relançada para que o processo termine com erro, permitindo que o orquestrador detecte a falha.

monitor.py

O arquivo foi organizado com uma combinação de funções auxiliares globais e uma classe principal chamada **MonitorSistema**.

As funções globais foram usadas porque algumas operações precisam ser acessadas por diferentes processos do sistema, principalmente pelos workers. Funções como **registrar_evento()**, **registrar_pedido_finalizado()** e **registrar_erro()** são chamadas pelos workers para informar ao monitor o que está acontecendo durante a execução. Essas funções não escrevem diretamente no arquivo de log.

```
def registrar_evento(fila_monitor: Optional[Queue], etapa: str, worker_id: int, pedido_id: str, evento: str, detalhes: str = ""):
    if fila_monitor is None:
        return

    mensagem = mensagem_base(TIPO_EVENTO)
    mensagem.update({
        "etapa": etapa,
        "worker_id": worker_id,
        "pedido_id": pedido_id,
        "evento": evento,
        "detalhes": detalhes
    })

    fila_monitor.put(mensagem)

def registrar_pedido_finalizado(fila_monitor: Optional[Queue], pedido):
    if fila_monitor is None:
        return

    mensagem = mensagem_base(TIPO_PEDIDO_FINALIZADO)
    mensagem["pedido"] = pedido.para_dict()

    fila_monitor.put(mensagem)

def registrar_erro(fila_monitor: Optional[Queue], detalhes: str):
    if fila_monitor is None:
        return

    mensagem = mensagem_base(TIPO_ERRO)
    mensagem["detalhes"] = detalhes

    fila_monitor.put(mensagem)
```

Elas apenas criam uma mensagem padronizada e enviam essa mensagem para a fila do monitor.

Essa abordagem evita que vários workers tentem escrever diretamente no mesmo arquivo ao mesmo tempo. Em um sistema com múltiplos processos, isso poderia gerar inconsistências ou mensagens misturadas. Com a fila, os workers apenas produzem eventos, enquanto o processo do monitor consome esses eventos e centraliza a escrita no arquivo .log e no relatório .json.

Além das funções de registro, o arquivo também possui funções auxiliares de formatação, como **cabecalho_mensagem()**, **formatar_evento()**, **formatar_pedido_finalizado()** e **formatar_erro()**. Essas funções foram separadas para evitar repetição de código e deixar o **monitor_worker** mais legível.

```
TIPO_EVENTO = "evento"
TIPO_PEDIDO_FINALIZADO = "pedido_finalizado"
TIPO_FINALIZAR = "finalizar"
TIPO_ERRO = "erro"

def timestamp() -> str:
    return datetime.now().strftime("%H:%M:%S.%f")[:-3]

def arquivo_json_de(arquivo_log: str) -> str:
    base, _ = os.path.splitext(arquivo_log)
    return base + ".json"

def escrever_linha_log(arquivo_log: str, linha: str):
    with open(arquivo_log, "a", encoding="utf-8") as arquivo:
        arquivo.write(linha + "\n")

def cabecalho_mensagem(mensagem: dict) -> str:
    return (f"[{mensagem.get('timestamp', timestamp())}] " f"[PID {mensagem.get('pid', '-')}] " f"[{mensagem.get('processo', '-')}]]")

def prefixo_monitor() -> str:
    return (f"[{timestamp()}] " f"[PID {os.getpid()}] " f"[{current_process().name}] " f"[MONITOR]")

def mensagem_base(tipo: str) -> dict:
    return {"tipo": tipo, "timestamp": timestamp(), "pid": os.getpid(), "processo": current_process().name}

def formatar_evento(mensagem: dict) -> str:
    linha = (f"{cabecalho_mensagem(mensagem)} "
            f"[{str(mensagem.get('etapa', '').upper())} {mensagem.get('worker_id', '-')}] "
            f"[{mensagem.get('pedido_id', '-')} - "
            f"{mensagem.get('evento', '')}]")

    detalhes = mensagem.get("detalhes")
    if detalhes:
        linha += f" | {detalhes}"

    return linha

def formatar_pedido_finalizado(mensagem: dict) -> str:
    pedido = mensagem["pedido"]
    return (f"{cabecalho_mensagem(mensagem)} " f"[PEDIDO_FINALIZADO] {pedido['id']} - {pedido['status']}")

def formatar_erro(mensagem: dict) -> str:
    return (f"{cabecalho_mensagem(mensagem)} " f"[ERRO] {mensagem.get('detalhes', '')}")
```

A função **monitor_worker()** representa o processo ativo do monitor. Ela fica em execução aguardando mensagens na fila de eventos. Quando recebe uma mensagem, identifica seu tipo e decide como registrá-la. Caso receba uma mensagem de finalização, encerra o loop, calcula o tempo total da execução e salva o relatório final em JSON.


```
def monitor_worker(fila_eventos: Queue, arquivo_log: str, arquivo_json: str, timestamp_inicio_iso: str, total_esperado: Optional[int] = None):
    timestamp_inicio = datetime.fromisoformat(timestamp_inicio_iso)

    inicializar_log(arquivo_log, timestamp_inicio)

    pedidos = []
    contadores_status = Counter()
    eventos_registrados = 0

    escrever_linha_log(arquivo_log, f"{prefixo_monitor()} Active Object iniciado")

    while True:
        mensagem = fila_eventos.get()

        if mensagem is None or mensagem.get("tipo") == TIPO_FINALIZAR:
            break

        eventos_registrados += 1
        tipo = mensagem.get("tipo")

        if tipo == TIPO_EVENTO:
            linha = formatar_evento(mensagem)

        elif tipo == TIPO_PEDIDO_FINALIZADO:
            pedido = mensagem["pedido"]
            pedidos.append(pedido)
            contadores_status[pedido["status"]] += 1
            linha = formatar_pedido_finalizado(mensagem)

        elif tipo == TIPO_ERRO:
            linha = formatar_erro(mensagem)

        else:
            linha = f"{prefixo_monitor()} Mensagem desconhecida: {mensagem}"

        escrever_linha_log(arquivo_log, linha)

    timestamp_fim = datetime.now()

    salvar_relatorio_json(arquivo_json=arquivo_json, timestamp_inicio=timestamp_inicio, timestamp_fim=timestamp_fim,
                        total_esperado=total_esperado, pedidos=pedidos, contadores_status=contadores_status, eventos_registrados=eventos_registrados)

    escrever_linha_log(arquivo_log, f"{prefixo_monitor()} Active Object finalizado")
    escrever_linha_log(arquivo_log, f"Total de pedidos finalizados: {len(pedidos)}")
    escrever_linha_log(arquivo_log, f"Contadores por status: {dict(contadores_status)}")
    escrever_linha_log(arquivo_log, f"Relatório JSON: {arquivo_json}")
```

```
def inicializar_log(arquivo_log: str, timestamp_inicio: datetime):
    with open(arquivo_log, "w", encoding="utf-8") as arquivo:
        arquivo.write("SISTEMA DE VENDAS - Iniciado em " f"{timestamp_inicio.strftime('%Y-%m-%d %H:%M:%S')}\n")
        arquivo.write("=" * 80 + "\n\n")

def salvar_relatorio_json(arquivo_json: str, timestamp_inicio: datetime,
                        timestamp_fim: datetime, total_esperado: Optional[int],
                        pedidos: list, contadores_status: Counter, eventos_registrados: int):

    tempo_total = (timestamp_fim - timestamp_inicio).total_seconds()

    relatorio = {
        "tempo_total_segundos": tempo_total,
        "timestamp_inicio": timestamp_inicio.isoformat(),
        "timestamp_fim": timestamp_fim.isoformat(),
        "total_esperado": total_esperado,
        "total_pedidos": len(pedidos),
        "contadores_por_status": dict(contadores_status),
        "eventos_registrados": eventos_registrados,
        "pedidos": pedidos
    }

    with open(arquivo_json, "w", encoding="utf-8") as arquivo:
        json.dump(relatorio, arquivo, indent=2, ensure_ascii=False)
```

A classe **MonitorSistema** fica responsável por controlar o ciclo de vida do monitor. Ela cria a fila de eventos, define os arquivos de saída, cria o processo `monitor_worker`, inicia esse processo, envia o sinal de finalização e depois permite consultar o relatório final.

Dessa forma, os workers apenas produzem eventos, enquanto o monitor consome esses eventos e centraliza a gravação no log e no JSON. Isso reduz acoplamento, evita escrita simultânea nos arquivos e mantém a lógica do monitor organizada.

```
class MonitorSistema:
    def __init__(self, arquivo_log: str = None, total_esperado: Optional[int] = None):
        self.arquivo_log = arquivo_log or "sistema_vendas.log"
        self.arquivo_json = arquivo_json_de(self.arquivo_log)
        self.timestamp_inicio = datetime.now()
        self.total_esperado = total_esperado
        self.fila_eventos = Queue()
        self.processo = Process(
            target=monitor_worker,
            args=(self.fila_eventos, self.arquivo_log, self.arquivo_json, self.timestamp_inicio.isoformat(), self.total_esperado),
            name="monitor-active-object")

    def iniciar(self):
        self.processo.start()

    def finalizar(self):
        if self.processo.is_alive():
            self.fila_eventos.put({"tipo": TIPO_FINALIZAR})
            self.processo.join()

        if self.processo.exitcode not in (0, None):
            raise RuntimeError(f"Monitor terminou com erro. exitcode={self.processo.exitcode}")

    def parar_forçado(self):
        if self.processo.is_alive():
            self.processo.terminate()
            self.processo.join(timeout=2)

            if self.processo.is_alive():
                self.processo.kill()

    def gerar_relatorio_final(self) -> dict:
        if not os.path.exists(self.arquivo_json):
            return {
                "tempo_total_segundos": (datetime.now() - self.timestamp_inicio).total_seconds(),
                "timestamp_inicio": self.timestamp_inicio.isoformat(),
                "timestamp_fim": datetime.now().isoformat(),
                "total_esperado": self.total_esperado,
                "total_pedidos": 0,
                "contadores_por_status": {},
                "eventos_registrados": 0,
                "pedidos": []
            }

        with open(self.arquivo_json, "r", encoding="utf-8") as arquivo:
            return json.load(arquivo)

    def exibir_resumo_final(self):
        relatorio = self.gerar_relatorio_final()

        tempo_total = relatorio["tempo_total_segundos"]
        total_pedidos = relatorio["total_pedidos"]
        total_esperado = relatorio.get("total_esperado")
        throughput = total_pedidos / tempo_total if tempo_total > 0 else 0.0

        print("SISTEMA CONCLUÍDO")
        print(f"Tempo total: {tempo_total:.2f} segundos")
        print(f"Pedidos finalizados: {total_pedidos}")

        if total_esperado is not None:
            print(f"Pedidos esperados: {total_esperado}")

        print(f"Throughput: {throughput:.2f} pedidos/s")
        print("\nContadores por status:")

        for status, quantidade in relatorio.get("contadores_por_status", {}).items():
            print(f" - {status}: {quantidade}")

        print(f"\nLog salvo em: {self.arquivo_log}")
        print(f"Relatório JSON salvo em: {self.arquivo_json}")

        return relatorio
```

orquestrador.py

A classe Orquestrador é responsável por coordenar a execução completa do sistema. No construtor da classe, são criadas as filas que conectam as etapas do pipeline (**fila_pedidos**, **fila_validados**, **fila_aprovados**, **fila_monitor**) e também é criado o monitor, que recebe o arquivo de log configurado e o total esperado de pedidos

```
class Orquestrador:
    def __init__(self, config: ConfiguracaoSistema = None):
        self.config = config or ConfiguracaoSistema()

        self.fila_pedidos = Queue(maxsize=self.config.tamanho_fila)
        self.fila_validados = Queue(maxsize=self.config.tamanho_fila)
        self.fila_aprovados = Queue(maxsize=self.config.tamanho_fila)
        self.monitor = MonitorSistema(arquivo_log=self.config.arquivo_log, total_esperado=self.config.total_pedidos_esperados())
        self.fila_monitor = self.monitor.fila_eventos
        self.processos: List[Process] = []
```

O método **iniciar_sistema()** inicia o monitor e cria os processos de trabalho. Primeiro são iniciados os clientes, depois os validadores, depois os financeiros e, por fim, os logísticos. Cada processo recebe os argumentos necessários, como identificador do worker, configuração, fila de entrada, fila de saída e fila do monitor

```
def iniciar_sistema(self):
    print("SISTEMA DE VENDAS PARALELO - INICIANDO")
    print(self.config)

    self.monitor.iniciar()

    print("\nINICIANDO CLIENTES...")
    for cliente_id in range(self.config.num_clientes):
        p = Process(target=cliente_worker, args=(cliente_id, self.config, self.fila_pedidos, self.fila_monitor), name=f"cliente-{cliente_id}")
        p.start()
        self.processos.append(p)

    print("\nINICIANDO VALIDADORES...")
    for validador_id in range(self.config.num_validadores):
        p = Process(target=validador_worker, args=(validador_id, self.config, self.fila_pedidos, self.fila_validados, self.fila_monitor), name=f"validador-{validador_id}")
        p.start()
        self.processos.append(p)

    print("\nINICIANDO FINANCEIROS...")
    for financeiro_id in range(self.config.num_financeiros):
        p = Process(target=financeiro_worker, args=(financeiro_id, self.config, self.fila_validados, self.fila_aprovados, self.fila_monitor), name=f"financeiro-{financeiro_id}")
        p.start()
        self.processos.append(p)

    print("\nINICIANDO LOGÍSTICOS...")
    for logistica_id in range(self.config.num_logisticos):
        p = Process(target=logistica_worker, args=(logistica_id, self.config, self.fila_aprovados, self.fila_monitor), name=f"logistica-{logistica_id}")
        p.start()
        self.processos.append(p)

    print(f"\n{len(self.processos)} processos de trabalho iniciados")
    print("Monitor iniciado\n")
```

O método **aguardar_conclusao()** controla o encerramento do pipeline. Ele primeiro espera os clientes terminarem. Depois envia sinais **None** para os validadores, indicando que não haverá mais pedidos na fila inicial. Quando os validadores terminam, envia sinais **None** para os financeiros. Depois faz o mesmo para os logísticos. Esse uso do valor **None** é importante porque funciona como um sinal de parada. Cada worker executa um loop infinito consumindo a fila. Quando recebe **None**, entende que deve encerrar.

```
def aguardar_conclusao(self):
    print("Aguardando conclusão...\n")

    self.join_e_verificar("cliente")

    for _ in range(self.config.num_validadores):
        self.fila_pedidos.put(None)
        self.join_e_verificar("validador")

    for _ in range(self.config.num_financeiros):
        self.fila_validados.put(None)
        self.join_e_verificar("financeiro")

    for _ in range(self.config.num_logisticos):
        self.fila_aprovados.put(None)
        self.join_e_verificar("logistica")

    print("\nTodos os processos de trabalho concluídos")
```

```
def processos_por_nome(self, prefixo: str) -> List[Process]:
    return [p for p in self.processos if p.name.startswith(prefixo)]

def join_e_verificar(self, prefixo: str):
    for p in self.processos_por_nome(prefixo):
        p.join()
        if p.exitcode != 0:
            raise RuntimeError(f"Processo {p.name} terminou com erro. exitcode={p.exitcode}")
```

O método **executar()** representa o fluxo completo da aplicação: iniciar o sistema, aguardar conclusão, finalizar o monitor e exibir o resumo final. Caso ocorra interrupção ou erro, o método chama **parar_sistema()** para encerrar os processos de forma segura

```
def executar(self):
    try:
        self.iniciar_sistema()
        self.aguardar_conclusao()
        self.monitor.finalizar()
        self.monitor.exibir_resumo_final()

    except KeyboardInterrupt:
        print("\n\nSistema interrompido")
        self.parar_sistema()
        self.monitor.parar_forcado()
        raise

    except Exception as e:
        print(f"\nErro: {e}")
        self.parar_sistema()
        self.monitor.parar_forcado()
        raise
```

```
def parar_sistema(self):
    print("\nParando sistema...")
    for p in self.processos:
        if p.is_alive():
            p.terminate()
            p.join(timeout=2)
            if p.is_alive():
                p.kill()

    if self.monitor is not None:
        self.monitor.parar_forcado()
```

main.py

É o ponto de entrada da aplicação e é responsável por preparar a configuração do sistema e iniciar a execução do orquestrador.

Inicialmente, o arquivo importa o Orquestrador e as configurações previamente definidas no módulo de configuração: **CONFIG_PADRAO**, **CONFIG_TESTE** e **CONFIG_PESADO**. Essas configurações são armazenadas em um dicionário chamado **CONFIGURACOES**, permitindo que o usuário escolha qual cenário deseja executar.

```
from src.controller.orquestrador import Orquestrador
from src.model.config import CONFIG_PADRAO, CONFIG_PESADO, CONFIG_TESTE

CONFIGURACOES = {
    "padrao": CONFIG_PADRAO,
    "teste": CONFIG_TESTE,
    "pesado": CONFIG_PESADO,
}

def criar_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        description="Executa o sistema de vendas paralelo."
    )
    parser.add_argument(
        "--config",
        choices=CONFIGURACOES.keys(),
        default="padrao",
        help="Configuracao base para executar.",
    )
    parser.add_argument("--clientes", type=int, default=None)
    parser.add_argument("--pedidos-por-cliente", type=int, default=None)
    parser.add_argument("--validadores", type=int, default=None)
    parser.add_argument("--financeiros", type=int, default=None)
    parser.add_argument("--logisticos", type=int, default=None)
    parser.add_argument("--tamanho-fila", type=int, default=None)
    parser.add_argument("--falha-validacao", type=float, default=None)
    parser.add_argument("--falha-financeira", type=float, default=None)
    parser.add_argument("--falha-logistica", type=float, default=None)
    parser.add_argument("--tempo-min", type=float, default=None)
    parser.add_argument("--tempo-max", type=float, default=None)
    parser.add_argument("--arquivo-log", type=str, default=None)
    return parser
```

```
def aplicar_sobrescritas(config, args):
    sobrescritas = {
        "num_clientes": args.clientes,
        "pedidos_por_cliente": args.pedidos_por_cliente,
        "num_validadores": args.validadores,
        "num_financeiros": args.financeiros,
        "num_logisticos": args.logisticos,
        "tamanho_fila": args.tamanho_fila,
        "taxa_falha_validacao": args.falha_validacao,
        "taxa_falha_financeira": args.falha_financeira,
        "taxa_falha_logistica": args.falha_logistica,
        "tempo_processamento_min": args.tempo_min,
        "tempo_processamento_max": args.tempo_max,
        "arquivo_log": args.arquivo_log,
    }

    for atributo, valor in sobrescritas.items():
        if valor is not None:
            setattr(config, atributo, valor)

    if hasattr(config, "_validar"):
        config._validar()

    return config
```

O arquivo utiliza a biblioteca `argparse` para permitir a configuração do sistema pela linha de comando. Com isso, é possível alterar valores como quantidade de clientes, pedidos por cliente, número de workers, tamanho das filas, taxas de falha, tempos de processamento e arquivo de log sem modificar diretamente o código-fonte.

A função **`criar_parser()`** define todos os argumentos aceitos pelo programa. Já a função **`aplicar_sobrescritas()`** recebe uma configuração base e substitui seus valores caso o usuário tenha informado novos parâmetros pelo terminal. Após aplicar essas alterações, a configuração é validada novamente para garantir que os valores informados sejam válidos.

A função **`main()`** representa o fluxo principal de inicialização. Ela lê os argumentos informados, copia a configuração escolhida, aplica as sobrescritas e cria uma instância do Orquestrador, chamando em seguida o método executar.

```
def main() -> None:
    args = criar_parser().parse_args()
    config = copy.deepcopy(CONFIGURACOES[args.config])
    config = aplicar_sobrescritas(config, args)
    Orquestrador(config).executar()
```

O bloco final:

```
if __name__ == "__main__":
    freeze_support()
    main()
```

Garante que a aplicação seja executada apenas quando o arquivo for chamado diretamente. O uso de **`freeze_support()`** é importante para compatibilidade com o módulo `multiprocessing`, principalmente em sistemas Windows.

Dessa forma, o **`main.py`** não contém a lógica de processamento dos pedidos, mas funciona como a camada inicial da aplicação, responsável por receber parâmetros, preparar a configuração e iniciar o sistema paralelo.

ESTRATÉGIA DE ENCERRAMENTO DOS PROCESSOS

Um ponto importante do projeto é a forma como os processos são finalizados. Como os workers ficam em loop esperando novos pedidos nas filas, eles precisam receber um sinal indicando que não haverá mais trabalho. Esse sinal é representado pelo valor `None`.

A finalização acontece em etapas:

- O orquestrador espera todos os clientes terminarem
- Envia um **`None`** para cada validador
- Espera todos os validadores terminarem
- Envia um **`None`** para cada financeiro
- Espera todos os financeiros terminarem
- Envia um **`None`** para cada logístico
- Espera todos os logísticos terminarem
- Finaliza o monitor

Essa ordem evita que uma etapa seja encerrada antes de consumir todos os pedidos produzidos pela etapa anterior. Por exemplo, os financeiros só recebem o sinal de parada depois que todos os validadores terminaram. Isso garante que nenhum pedido validado fique preso na fila sem ser processado.

RESULTADOS

Após a execução do sistema, o monitor registrou a finalização de 16 pedidos. Esse total corresponde aos pedidos produzidos pelos clientes durante a simulação. O processamento foi realizado por múltiplos processos, incluindo produtores, validadores, financeiros e logísticos, todos se comunicando por meio de filas.

Ao final da execução, o sistema apresentou a seguinte distribuição de status:

- 10 pedidos foram entregues com sucesso
- 2 foram rejeitados na validação
- 2 foram rejeitados na etapa financeira
- 2 apresentaram falha na entrega

Dessa forma, 62,5% dos pedidos chegaram ao estado final de entrega, enquanto os demais foram encerrados em etapas intermediárias ou apresentaram falha logística.

Os resultados obtidos demonstram que o pipeline funcionou corretamente. Pedidos rejeitados na validação não seguiram para a etapa financeira, pedidos rejeitados financeiramente não foram enviados à logística, e apenas pedidos aprovados financeiramente chegaram à etapa de entrega.

O log também evidencia o comportamento paralelo do sistema. Diferentes clientes produziram pedidos em horários muito próximos, enquanto múltiplos validadores, financeiros e logísticos processaram pedidos simultaneamente. Isso mostra que o sistema utilizou concorrência para permitir que diferentes etapas do fluxo trabalhassem ao mesmo tempo.

A execução completa durou aproximadamente 10,7 segundos, resultando em um throughput aproximado de 1,5 pedido por segundo. Esse resultado reforça a proposta do projeto, que é demonstrar o uso de processos, filas, worker pools e pipeline em uma aplicação prática de programação paralela.

```
Todos os processos de trabalho concluídos  
SISTEMA CONCLUÍDO
```

```
Tempo total: 10.68 segundos
```

```
Pedidos finalizados: 16
```

```
Pedidos esperados: 16
```

```
Throughput: 1.50 pedidos/s
```

```
Contadores por status:
```

- Rejeitado na Validação: 2
- Rejeitado Financeiramente: 2
- Falha na Entrega: 2
- Entregue: 10

```
Log salvo em: sistema_vendas.log
```

```
Relatório JSON salvo em: sistema_vendas.json
```

Análise do throughput

O throughput é uma métrica utilizada para medir a vazão do sistema. No projeto, essa métrica é calculada dividindo o total de pedidos finalizados pelo tempo total de execução.

Na execução realizada, o sistema finalizou 16 pedidos em aproximadamente 10,7 segundos, resultando em um throughput de cerca de 1,5 pedido por segundo. Esse valor mostra que o sistema conseguiu processar múltiplos pedidos em paralelo, pois diferentes etapas do pipeline estavam ativas ao

mesmo tempo.

O throughput pode variar de acordo com a configuração utilizada, por exemplo, ao aumentar a quantidade de workers em uma etapa pode melhorar o desempenho caso essa etapa seja um gargalo. Se muitos pedidos ficam aguardando entrega, aumentar o número de workers logísticos pode elevar a vazão

do sistema, da mesma forma, se os pedidos se acumulam antes da validação, aumentar o número de validadores pode melhorar o resultado.

Entretanto, o aumento no número de workers não garante crescimento infinito do throughput. O desempenho final depende do equilíbrio entre as etapas do pipeline, do tamanho das filas, do tempo de processamento simulado e das taxas de falha. Se uma etapa continuar mais lenta que as demais, ela limitará a vazão total do sistema.

CONCLUSÃO

Este projeto demonstrou a aplicação prática de conceitos de programação paralela e concorrente por meio da simulação de um sistema de vendas. A utilização de processos independentes, filas de comunicação e workers permitiu dividir o processamento dos pedidos em etapas, seguindo um modelo de pipeline.

Durante a execução, foi possível observar que clientes, validadores, financeiros e logísticos atuaram de forma simultânea, enquanto o monitor centralizou os registros e gerou os resultados finais. Com isso, o sistema conseguiu processar os pedidos de maneira organizada, evitando que múltiplos processos acessassem diretamente os mesmos arquivos de saída.

Portanto, o projeto cumpriu seu objetivo ao demonstrar o uso de multiprocessing, filas, worker pools, monitoramento e encerramento controlado de processos em um cenário prático.